VOICE RECOGNITION FOR FAIL SAFE COMMANDS

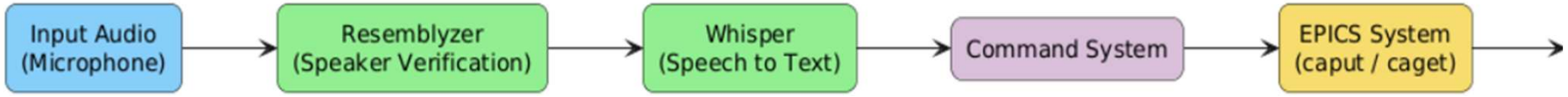
Presenter : **Mayank Rupesh Raut**
Department: **IADD,BARC**
Guide: **Mr. Arindam Basu**

PROBLEM STATEMENT

- Commands must be validated and executed securely to avoid incorrect operation
- Implement robust speaker verification to ensure secure system access
- Develop a reliable voice-based control system for accurate command execution
- Integrate the system with the EPICS environment for command execution
- There is a need for a secure, efficient, and voice-based interface for hands-free operation
- Enable hands-free operation and increased accessibility and reduced cognitive load

OBJECTIVE

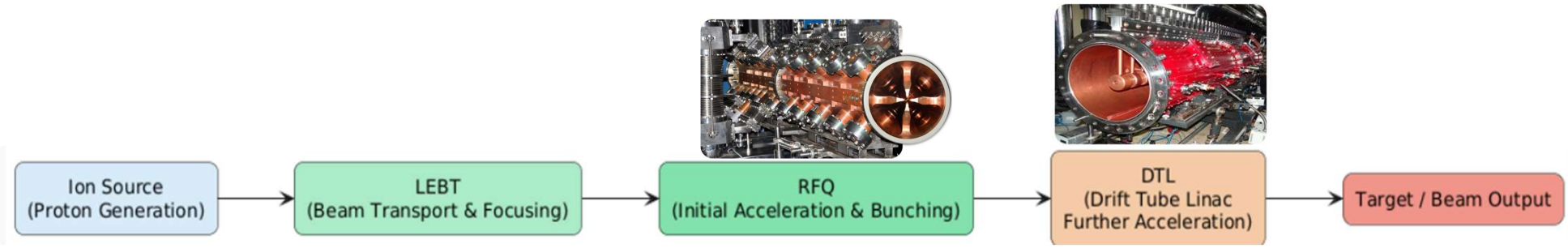
- Advanced Voice Recognition Development
- Speaker Authentication & Access Control
- Expanded Fail-Safe Command Set
- Control System Integration
- Implement safe shutdown sequences instead of abrupt stops.
- Logging, Auditing & Compliance
- Graphical User Interface & Monitoring



.Pipeline for voice recognition for fail safe commands

INTRODUCTION TO LEHIPA

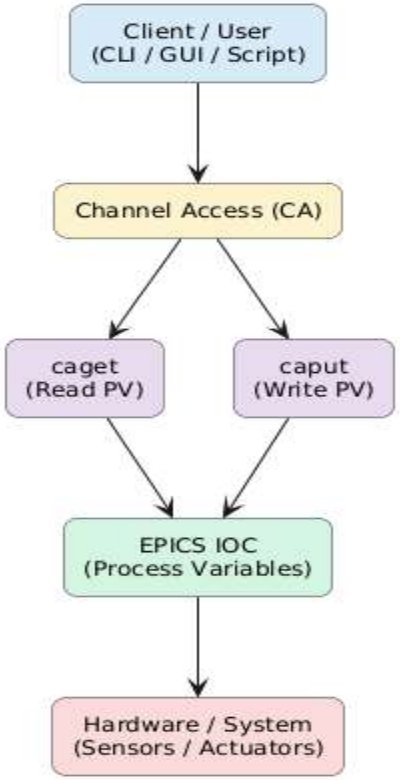
- LEHIPA (Low Energy High Intensity Proton Accelerator) is a 20Mev proton linear accelerator.
- It is the front end part for ADS (Accelerator Driven Subcritical) reactor system.
- Lehipa consists :
  - •Ion Source → Generates 50keV H^+ (proton beam)
  - •**LEBT** (Low Energy Beam Transport) → Beam focusing, steering and transport
  - •**RFQ** (Radio Frequency Quadrupole) → Initial acceleration, focusing and bunching (Accelerates the beam from 50keV to 3MeV)
  - •**MEBT** (Medium Energy Beam Transport) → Beam focusing and bunching
  - •**DTL** (Drift Tube Linac) → Further acceleration to 20MeV energy



Flow structure of LEHIPA

What is EPICS ?

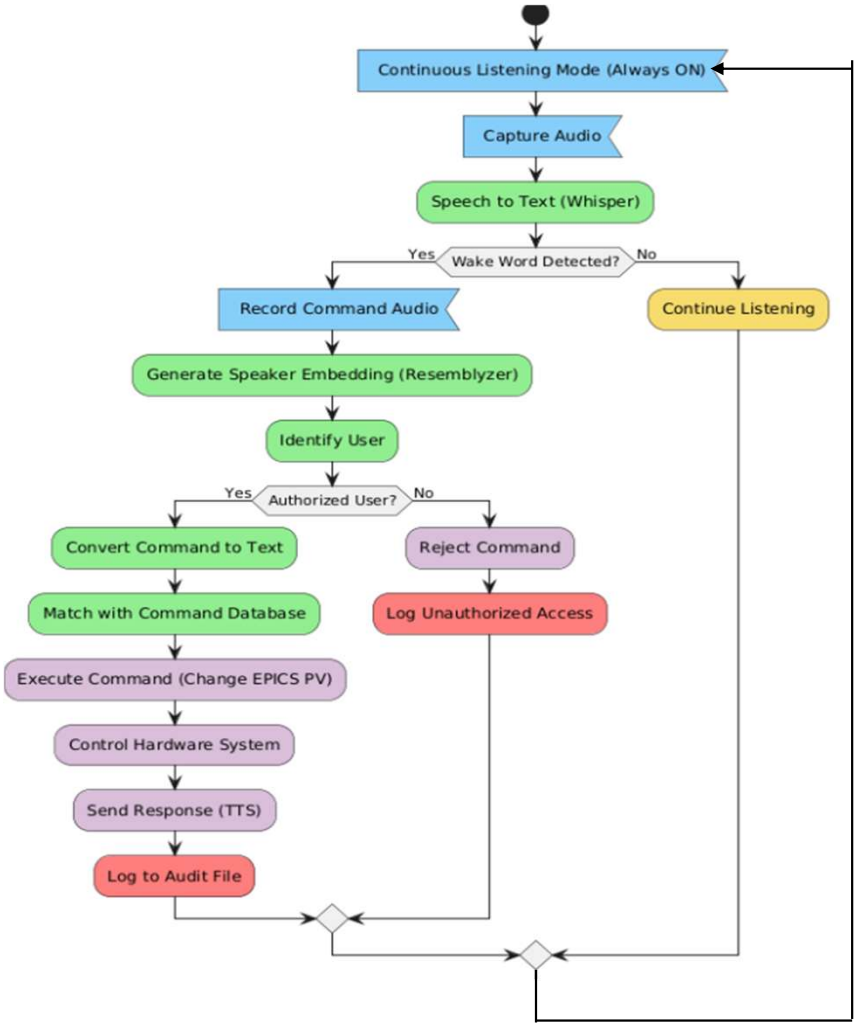
- **EPICS** (Experimental Physics and Industrial Control System) distributed control system framework developed collaboratively by major accelerator labs all over the world.
- Each field operation of the system is linked to EPICS process variables (**PVs**)
- Commands are executed by manipulating the PV values, which helps in control and monitoring the system
- EPICS Channel Access allows communication between different sub-systems of LEHIPA
- EPICS uses CA to manipulate the PVs, e.g.,
  caput → to write PV values
  caget → to read PV values



Flow chart of EPICS interface of Voice Based Authentication system (VBAS)

System Flow

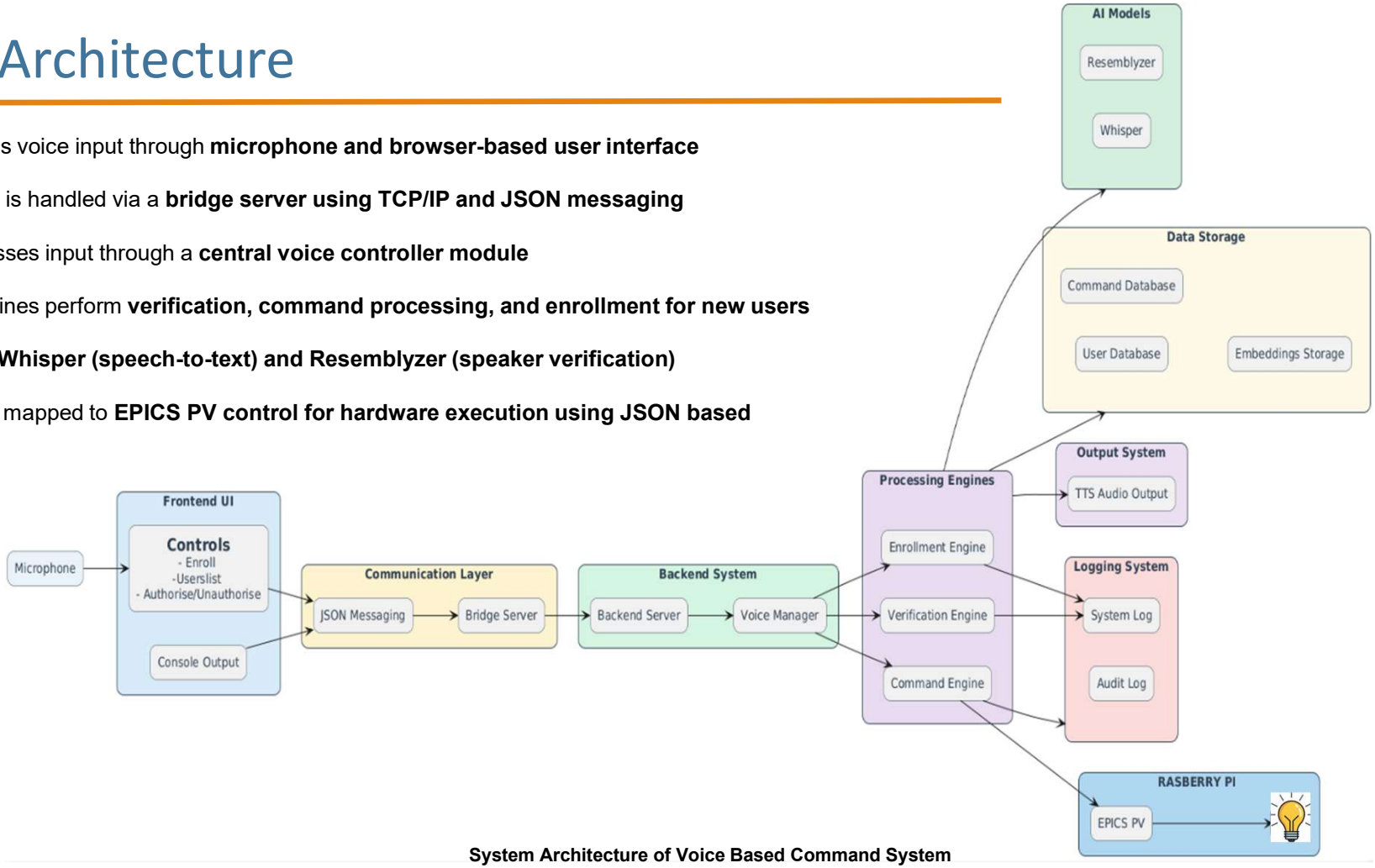
- System continuously listens for audio input
- Speech is converted to text when wake word is detected
- If wake word is absent, system continues listening
- If detected, command audio is recorded and then user is verified
- Unauthorized users are rejected and reported
- Authorized commands are processed and verified with the database
- User access is validated based on Assigned Role (Admin/User)
- EPICS PV values are updated to control hardware
- System gives audio feedback, logs and returns to listening



Flow chart for Voice Based Command System

System Architecture

- System captures voice input through **microphone and browser-based user interface**
- Communication is handled via a **bridge server using TCP/IP and JSON messaging**
- Backend processes input through a **central voice controller module**
- Processing engines perform **verification, command processing, and enrollment for new users**
- Integrated with **Whisper (speech-to-text) and Resemblyzer (speaker verification)**
- Commands are mapped to **EPICS PV control for hardware execution using JSON based database**



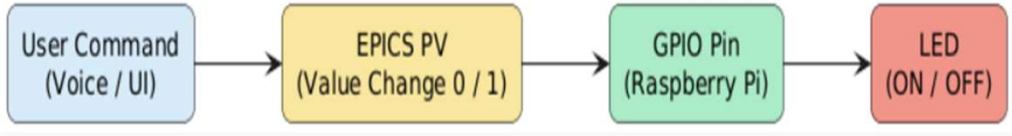
System Architecture of Voice Based Command System

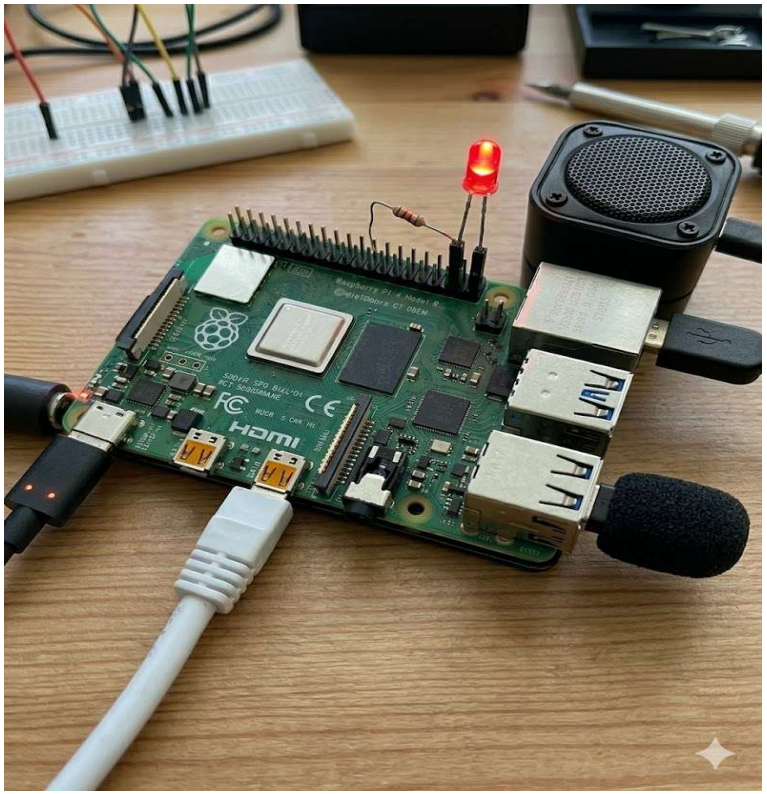
TECHNOLOGY STACK

- **Programming Language**: Python + JS
- **Speech Recognition**: Whisper
- **Speaker identification**: Resemblyzer
- **Audio Processing**: Sounddevice, Librosa
- **Control Interface**: EPICS (simulated)
- **Platform**: Linux (Ubuntu)

Hardware – Raspberry PI 4B

- System deployed on **Raspberry Pi 4B for execution**
- Microphone connected for **voice input capture**
- LED connected to **GPIO pin to demonstrate hardware interface**
- LED state changes (**ON/OFF**) based on command execution via **EPICS PV**
- Speaker connected for **audio feedback using audio output**
- Enables **complete interaction loop**



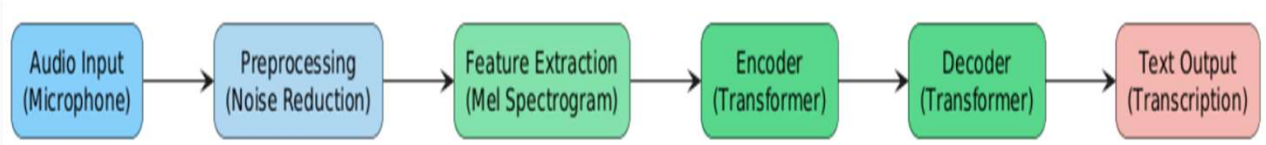


System setup on Raspberry Pi 4B

MODELS USED - WHISPER

- **Whisper Tiny (Speech Recognition)**
  - •Converts **audio input into text transcription**
  - •Model size: 75 MB
  - •Parameters = 39M
  - •Total data: ~680,000 hours of audio
  - •Transformer-based encoder-decoder model
  - •Trained on large-scale multilingual speech data
  - •Selected due to low computational requirement and CPU compatibility
  - •Handles transcription with acceptable accuracy

- **Features extracted[whisper]:**
  - •Frequency
  - •Temporal
  - •Phonetics
  - •Linguistics

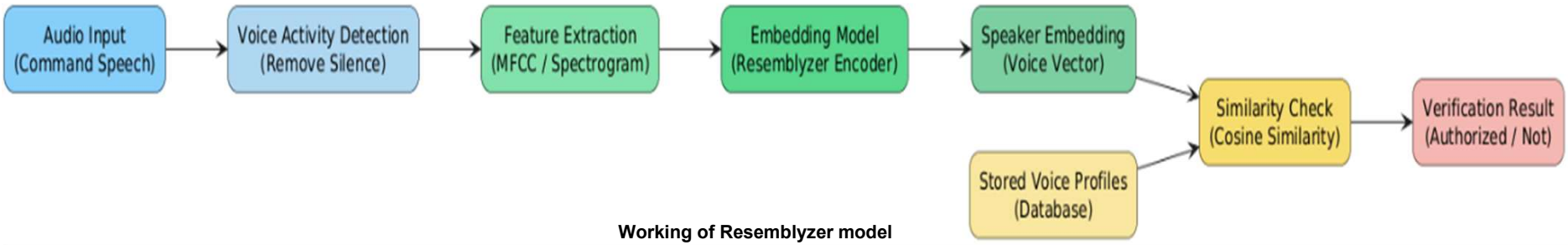


Working of Whisper model

MODELS USED - Resemblyzer

- **Resemblyzer (Speaker Verification)**
  - •Generates **speaker embeddings from voice input**
  - •Model size: 100 MB
  - •1000 hours of clean speech data
  - •Embeddings capture unique vocal characteristics
  - •Uses cosine similarity to compare identities
  - •Supports fast and efficient speaker matching

- **Features extracted [resemblyzer]:**
  - •Pitch
  - •Tone
  - •Voiceprint
  - •Style



Working of Resemblyzer model

Other Models tested

- **Vosk**
  - Small Size-50MB ; Offline model for RPi
  - Faster but **Lower transcription accuracy**
  - Vosk models on a dataset rich in accents and complex sentences
  - The accuracy rate as compared to whisper was 30%

- **SpeechBrain**
  - Open source All-in-one speech toolkit (speech-to-text, speaker recognition, TTS, enhancement, translation, etc)
  - Not optimized for real-time
  - Good for research and prototype development
  - The avg. word error rate was more then 0.7 for each command

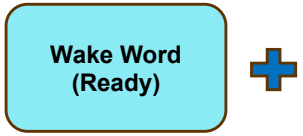
- **Other Whisper Variants (Base, Small, etc.)**
  - Size – (Base = 74M; SMALL = 244M)
  - Higher computational requirements
  - Increased latency on Raspberry Pi

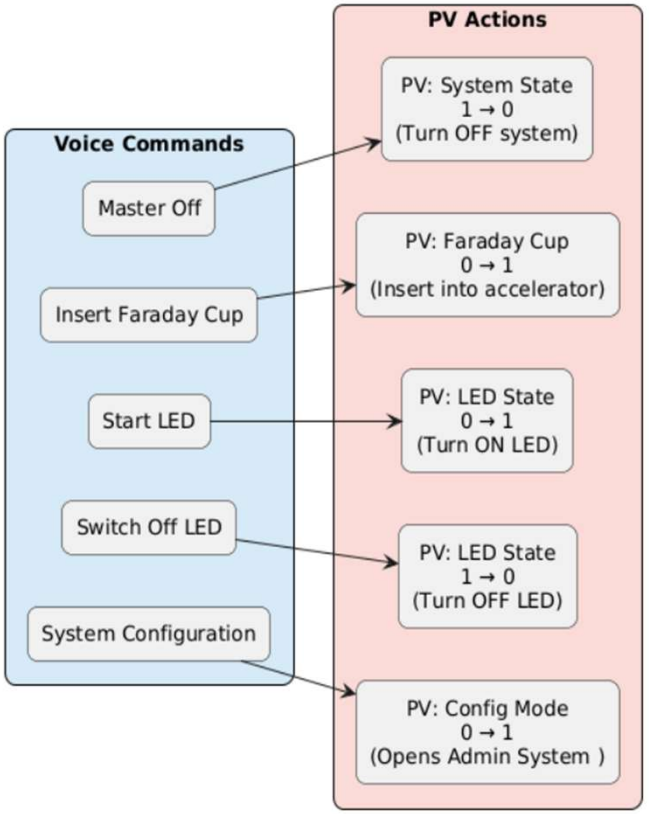
System Development

- Initial model development and testing performed on **Megh server environment**
- Megh machine specs: **12GB RAM, Intel Xeon Gold 6230R CPU @ 2.10GHz, ~100GB storage**
- Models tested: **Whisper, Vosk, SpeechBrain, Resemblyzer**
- Audio recorded on **local system (with microphone)** and transferred to Megh
- Voice system tested on **multiple users with different command inputs**
- Whisper identified as the most **feasible and accurate model**
- Final system deployed on **Raspberry Pi 4B with microphone and LED for testing**

Commands for Fail-Safe System

- **Wake Word Activation**
  System processes commands only after detecting a **wake word**
- **Similarity Threshold Matching (0.80)**
  Ensures commands are executed only when **closely matching predefined commands**
- **Predefined Command Set (Whitelisting)**
  Only **safe and allowed commands** are executed
- **Audio Handling & Processing**
  Filters input using **continuous listening and controlled activation**
- **Speaker Verification (Security Layer)**
  Only **authorized users** can execute commands
- **System Configuration Command (Admin Access)**





Commands and PV changes in the fail safe system

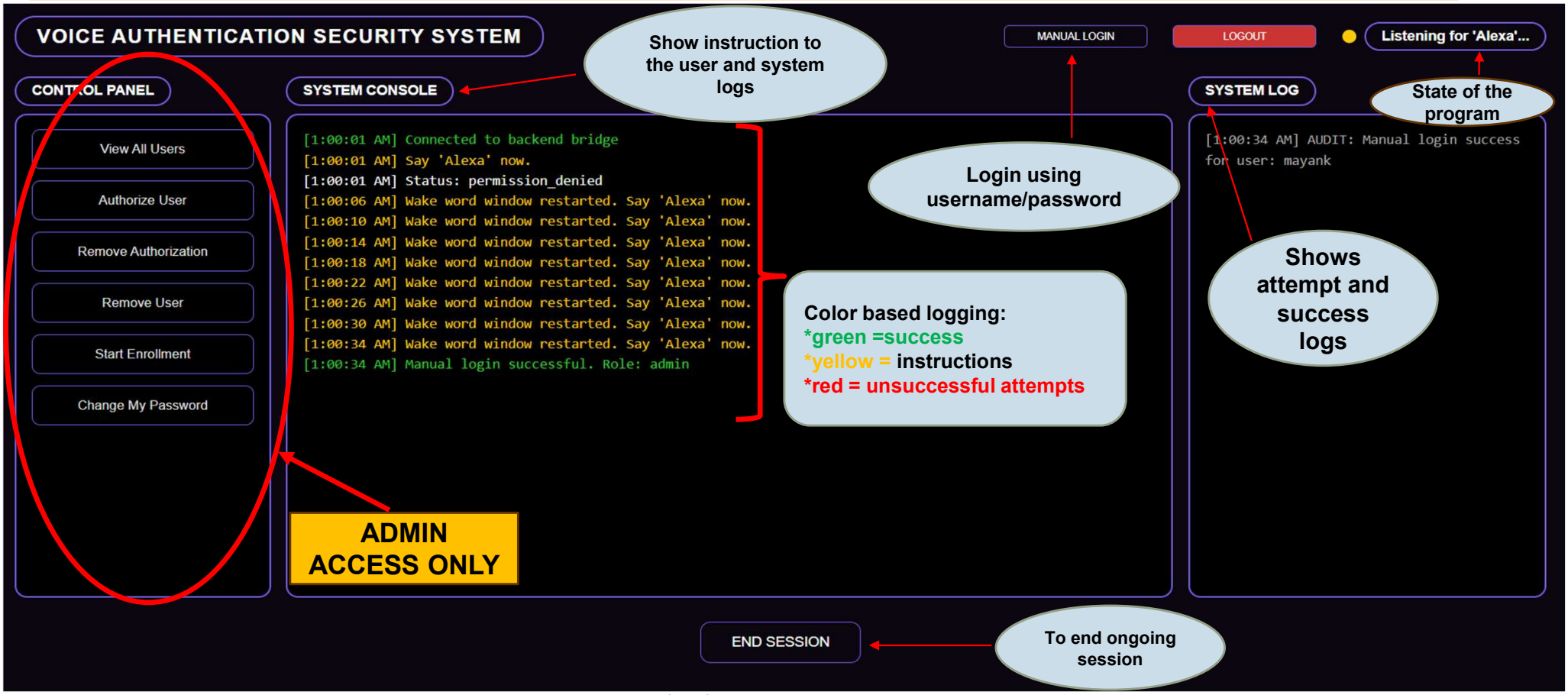
Role Based System

- System implements **two** roles**: Admin and User**
  - Access to system operations is controlled based on **user role and authorization**
  - Only **verified and authorized users** are allowed to execute commands
  - Prevents **unauthorized access and unintended system operations**
  - Role-based design ensures **secure and structured control of the system**

1. **Admin Role – User management**
   - Enroll new users by **capturing and storing voice embeddings**
   - Authorize or remove users from the system
   - Execute all predefined system commands
   - Manage access control and system-level operations

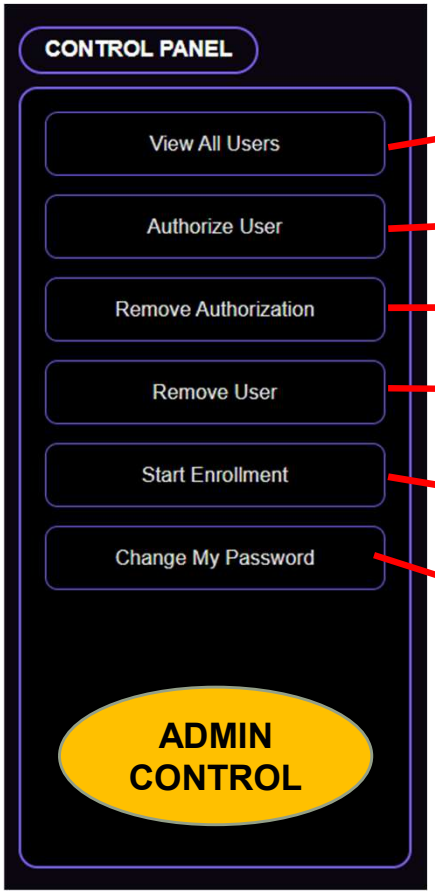
2. **User Role**
   - Execute commands only if **authorized and verified**
   - Limited access to system functionality
   - Cannot enroll or manage other users

Web based UI Interface



UI interface for voice based command system

UI Interface -Control Panel



Give the list of all the users

authorizes enrolled users and assigns roles

Remove authorization and can change roles

Removes user from the database

Starts enrollment

Once a user is given the role of admin they can set there own key

Limitations

- Whisper may produce **minor transcription errors (WER)**, especially for unclear speech
- Speaker verification accuracy decreases with **short or low-quality audio samples**
- System performance is limited by **CPU constraints of Raspberry Pi**
- Background noise can impact **speech recognition and verification accuracy**
- Requires proper **threshold tuning for reliable speaker verification**

Future Scope

- Improve speech recognition accuracy using larger or fine-tuned Whisper models
- Enhance speaker verification with advanced embedding models and better datasets
- Implement noise reduction and filtering techniques for robust performance
- Expand system to support dynamic and natural language commands (NLP integration)
- Optimize system performance using GPU acceleration or edge AI devices
- Develop scalable architecture for multi-user environments

Conclusion

- Developed a **secure Voice-based Command System** for control operations
- Successfully integrated **Speech Recognition and Speaker Verification models**
- Enabled **interaction with EPICS control system using PVs**
- Implemented **Role-based access control**
- Developed **end-to-end pipeline from voice input to hardware output**
- System demonstrates a **practical and scalable approach for VOICE-CONTROLLED automation**

Acknowledgements

I would like to express my sincere gratitude to:

- **Mr. Arindam Basu Sir (Mentor)**
- **Namrata Ma’am**
- **Bhumesh Sir**
- **Santosh Sir**
- **Deepak Sir**

for their **valuable guidance, support, and encouragement** throughout the development of this project.